用AI写PPT，没想到效果这么好啊

腾讯技术工程 2025年12月29日 01:36 广东 19人



作者： deyuankong

> "明天有个汇报，今晚必须交最终稿 PPT..." 这场景熟悉吗？ 年底到了，又到了写 PPT 的"煎熬季"，也许你正在为写 PPT 而绞尽脑汁、通宵达旦。这一次，你可以给 CodeBuddy 一个写 PPT 的机会，本文将详细地手把手实操，助力大家写好 PPT。

一、写PPT的痛点与需求

1.1 四大痛点，你中了几个？

作为职场上混的人免不了要写 PPT，下面的痛点你中了几个？

- **突然通知"明天汇报"，你却连模板都没有，缺模版**

1、没统一模板或缺乏各种思维模型的表达结构的模版，每次从零开始

2、网上免费的太丑，付费的太贵（99-299元/套）

3、想要简洁大气？自己设计不来

- **知道要说什么，但不知道怎么说，缺思路**

1、业务背景、技术难点、解决方案...该怎么排序？

2、数据一堆，不知道强调哪个

3、想突出亮点，逻辑却一团乱

- **好模板收费，专业 PPT 美化师更昂贵，缺资源**

1、专业模板网站：99-299元/套

2、设计师定制：费用昂贵，高达 500-2000元

3、买了还要学怎么用

- **本来就忙，哪来时间慢慢磨 PPT？没时间**

1、白天忙业务，晚上赶 PPT

2、改来改去，还是不满意

3、想要精美图表？没时间一点点调

面对上面的痛点， 于是你花了 2 小时找模板，最后还是选择" 职场土味商务风"、写了 50 页晋升 PPT，评委一句"所以你做了啥？"、PPT 美化要么花钱，要么凑合用、晚上熬夜到凌晨 3 点，第 2 天 顶着黑眼圈去汇报，心惊胆战，如履薄冰。这一切都别担心，CodeBuddy 来帮你！

1.2 CodeBuddy解决方案：Skills

当前，CodeBuddy Skills 为大家提供了一个优雅的 AI 解决方案——— 用 document-skills pptx 技能帮你搞定写 PPT ， Skills 一句话理解就是给为 AI Agent 配备人类现实世界所需的「技能包」，Skills 提前封装好工作流、工具、脚本等依赖信息，专注处理具体任务，并给出确定的结果，就像我们学习一门技术的时候，不会就赋能培训，具备从事专业能力。

今天，考虑我们写 PPT 的受众有很多非专业开发者，我知道大家非常痛，很需要帮助，CodeBuddy 没有忘记，今天将以手把手的实操，助力大家写好 PPT，希望给到大家一些帮助，也希望大家在汇报过程顺利、晋升成功。

二、快速入门

2.1 装CodeBuddy

目前 CodeBuddy IDE & CodeBuddy Code 均支持 Skills（内网版插件近期迭代上线），在使用之前，需安装 CodeBuddy IDE 或 CodeBuddy Code，两者任选其一。非专业开发者，建议使用 CodeBuddy IDE，专业开发者、DevOps、运维工程师以及 Claude Code 用户建议使用 CodeBuddy Code 。

- **安装 CodeBuddy IDE，**下载安装包

-官网地址：https://copilot.tencent.com/ide

- **安装 CodeBuddy Code**，打开终端 & 执行，一行命令搞定

-安装方式：npm install -g @tencent-ai/codebuddy-code & codebuddy update

2.2 技能包安装与初次体验

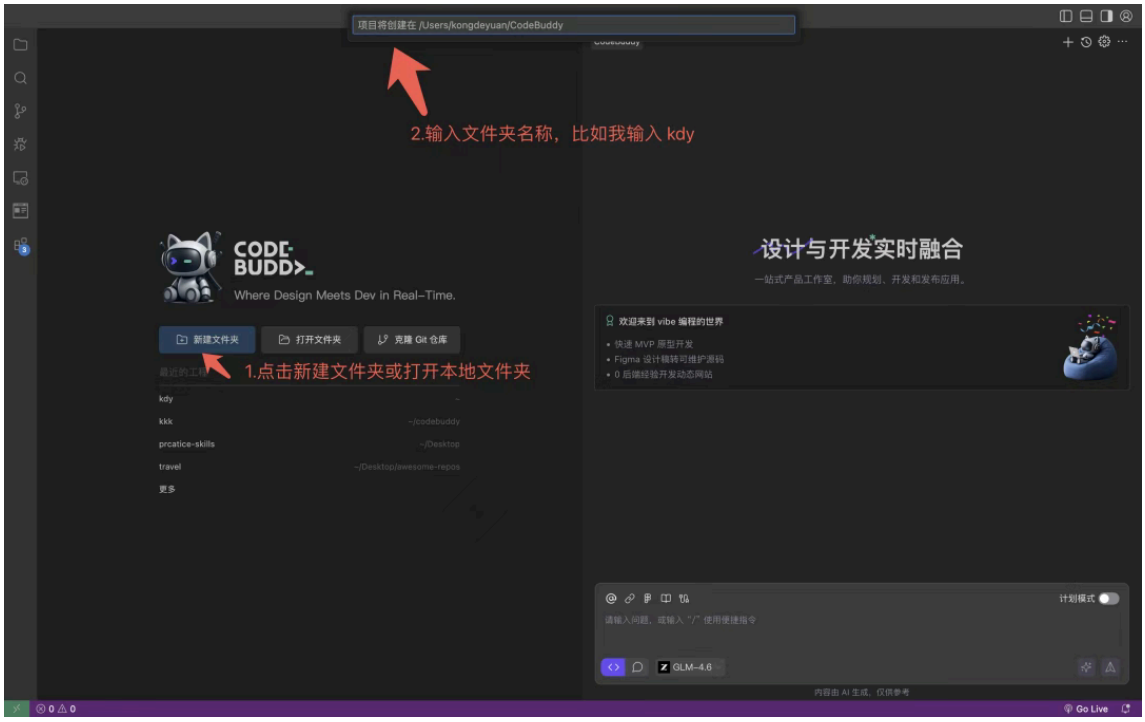
由于我们是需要写的是 PPT ， 因此需要安装具有 ppt 演示文稿创建、编辑和分析的Skills 技能包，目前在 Github 上已经有了 Anthropics 官方出品的 Skills 技能包，仓库地址：**GitHub - anthropics/skills: Public repository for Skills**，我们只需要在 CodeBuddy 中进行应用配置即可使用，其中 document-skills 技能包包含 docx、ppt、pdf、xlsx 4 种 Skill 配置，满足 Word、PPT、PDF、Excell 文件类型的创建、编辑和分析，接下来我们进行安装体验。

2.2.1 在 CodeBuddy IDE 中装 document-skills 技能包

● 非专业开发者，适合纯技术小白

第1步：下载安装包到本地并进行双击解压：document-skills.zip

第2步：打开 CodeBuddy IDE，创建好本地文件夹

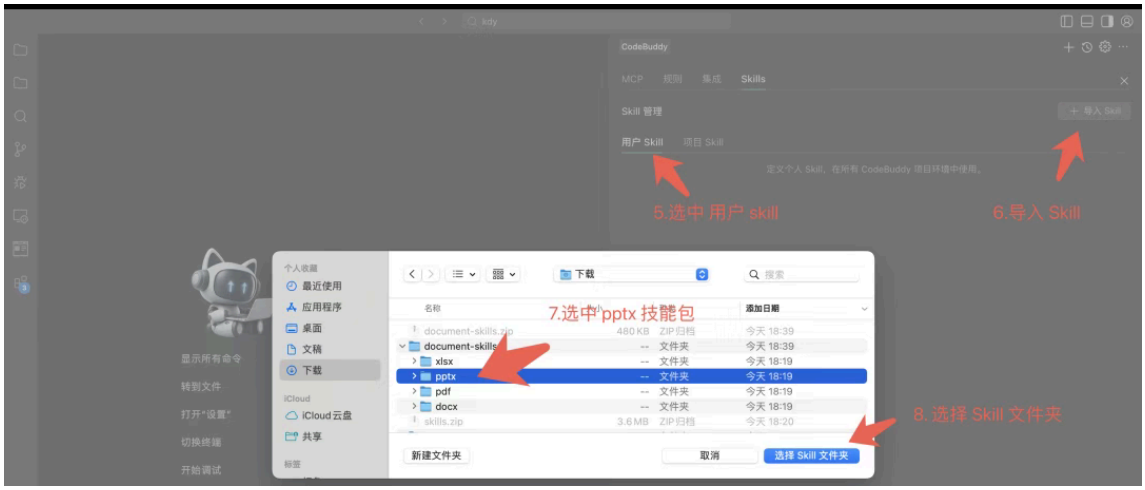


第3步： 进入配置页，找到 Skill 配置路径

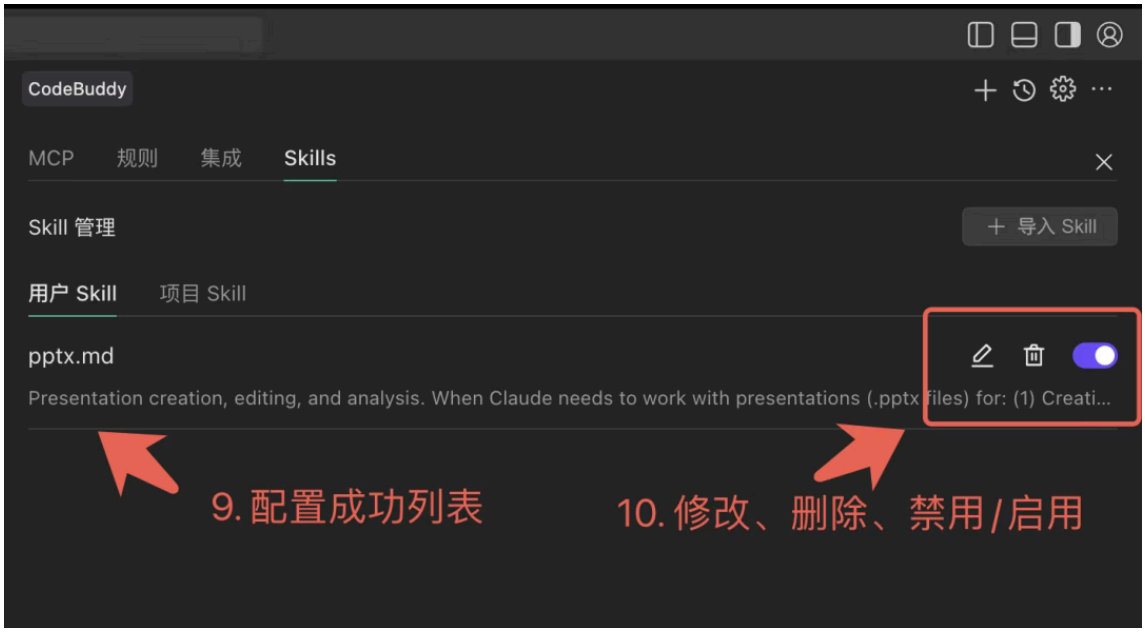
其中用户Skill是为全局配置，一次配置，任意文件夹都可用，建议选择；项目级Skill配置，仅支持在当前项目文件夹可用。



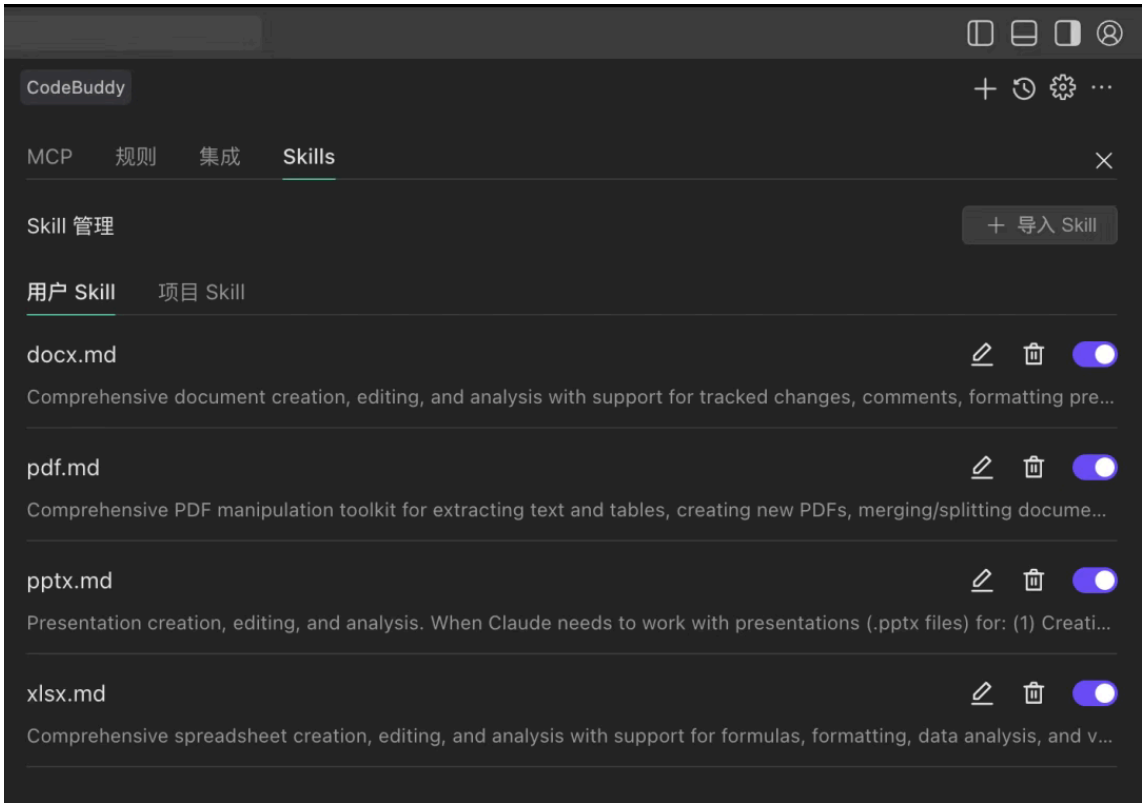
第 4 步：进行导入，我进行导入用户级 Skill



第 5 步：检查是否配置成功，成功后有如下列表



第 6 步（可选）：考虑 PPT 场景，建议导入 xlsx、pdf、docxSkills进行安装，操作方式如上，这些 Skills 完全覆盖了办公场景的 Excell、Word、PPT、PDF 等类型的数据处理。



第7步:初步体验skills

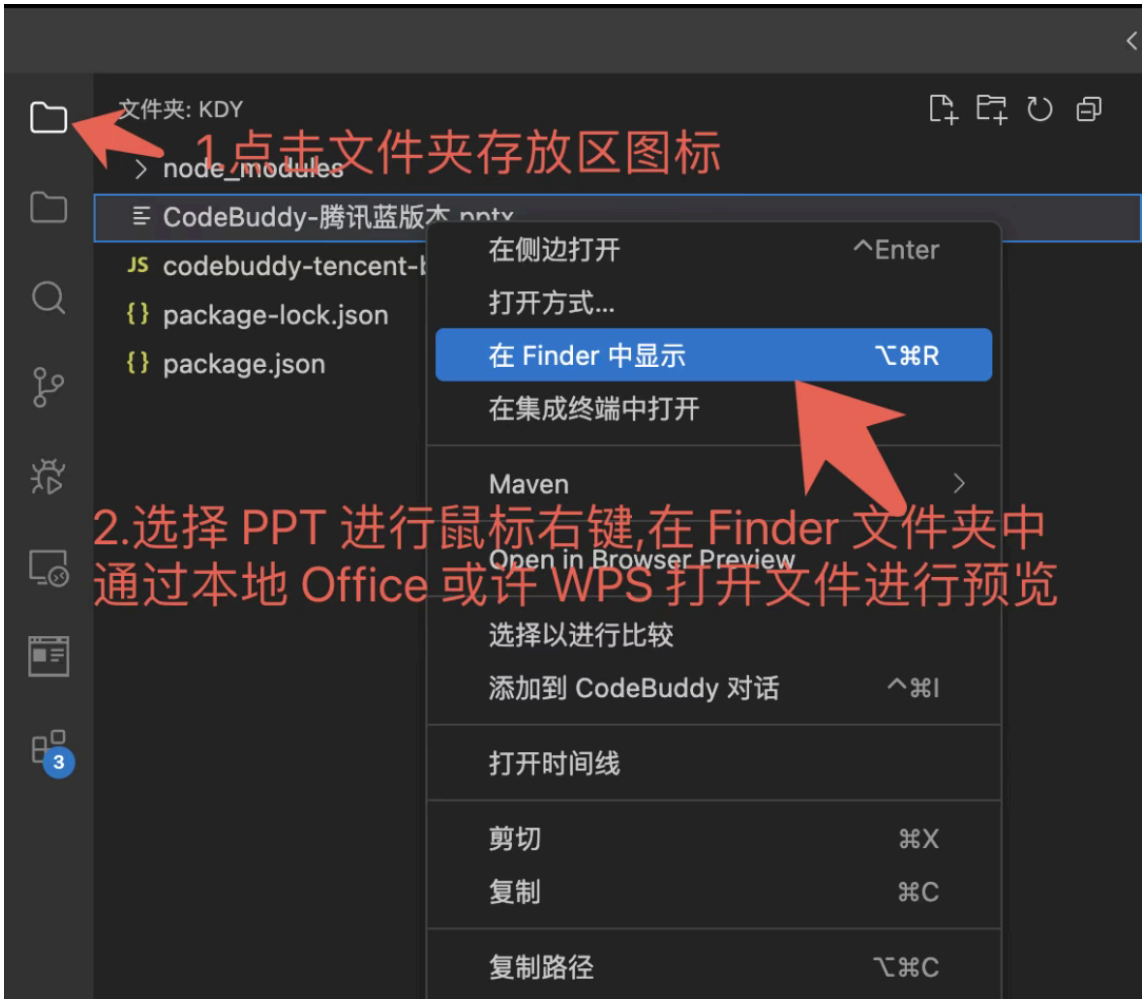
在CodeBuddy对话框中选择Agent模式、选择模型以及输入以下Prompt

```
面向非专业开发者，基于 https://copilot.tencent.com 网站内容，写一份介绍 PPT ，背景色采用 腾讯蓝 #0
```

示例图：与 CodeBuddy Agent 交互图



- **第8步: 初步体验 skills**



- **第9步：**查看 PPT 效果图：完整 PPT 见**CodeBuddy-腾讯蓝版本.pptx**

示例图：CodeBuddy 生成的 PPT 截图 1



示例图：CodeBuddy 生成的 PPT 截图 2



示例图：CodeBuddy 生成的 PPT 截图 3



- **专业开发者**

专业开发者，可以通过在 CodeBuddy IDE/Code 中进行配置 Skills 目录：

```
一、选择配置级别
#1.在项目中配置项目 Skills
.codebuddy/skills/
#2.在本地配置用户级 Skills
~/.codebuddy/skills/
#3.例如在 用户级配置 Skills
mkdir - p ~/.codebuddy & cd ~/.codebuddy

二、克隆代码到用户级目录
git clone https://github.com/anthropics/skills.git
```

2.2.1 在 CodeBuddyCode中装 document-skills 技能包

在 CodeBuddy Code 中 安装 document-skills 可按照上述 CodeBuddy IDE 中专业开发者的方式，也可以按照如下方式安装

- **第 1 步：打开终端&输出codebuddy**



- **第2步：执行安装** document-skills，分别执行如下两条命令

```
/plugin marketplace add anthropics/skills
/plugin install document-skills@anthropic-agent-skills
```

- **第3步： 初次体验 Skills ， 输入 prompt**

```
面向非专业开发者，基于 https://copilot.tencent.com 网站内容，写一份介绍 PPT ，背景色采用 腾讯蓝 #0
```

示例图：在 CodeBuddy 中 PPT Skill 进行创建 PPT Todo 进度项



- **第4步：效果图，待运行完todo事项，可以直接进行open.打开文件所在目录进行预览PPT。**

示例图：CodeBuddy 生成的 PPT 封面



示例图：CodeBuddy 生成的 PPT



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
689篇原创内容
公众号

三、场景实战

CodeBuddy Skills PPT Skill 支持在无模板的场景下实现PPT 生成，以下为4 个场景进行实战展示。

3.1 无模板场景下的PPT实战

3.1.1 输入 Prompt

```
我想开发一个酷炫的音乐APP，现在需要采用 skill 输出高保真的原型图，最终需要 PPT 保留，请通过以下方式帮我
1、用户体验分析：先分析这个 App 的主要功能和用户需求，确定核心交互逻辑。
2、产品界面规划：作为产品经理，定义关键界面，确保信息架构合理。
3、高保真 UI 设计：作为 UI 设计师，设计贴近真实 iOS/Android 设计规范的界面，使用现代化的 UI 元素，使
4、HTML 原型实现：使用 HTML + Tailwind CSS（或 Bootstrap）生成所有原型界面，并使用 FontAwesome（
5、每个界面应作为独立的 HTML 文件存放，例如 home.html、profile.html、settings.html 等。
index.html 作为主入口，不直接写入所有界面的 HTML 代码，而是使用 iframe 的方式嵌入这些 HTML 片段，并
真实感增强：
界面尺寸应模拟 iPhone 16 Pro，并让界面圆角化，使其更像真实的手机界面。
使用真实的 UI 图片，而非占位符图片（可从 Unsplash、Pexels、Apple 官方 UI 资源中选择）。
添加顶部状态栏（模拟 iOS 状态栏），并包含 App 导航栏（类似 iOS 底部 Tab Bar）。
请按照以上要求生成完整的 HTML 代码，并确保其可用于实际开发。
```

3.1.2 检验效果

以下为 CodeBuddy PPT Skill 运行生成 HTML & PPT图，详见**音乐APP原型设计_终版.pptx**

示例图：CodeBuddy PPT Skill 运行生成 HTML 交互图



示例图：CodeBuddy PPT Skill 运行生成 PPT 及服务启动图



示例图：CodeBuddy PPT Skill 生成 HTML 效果图1



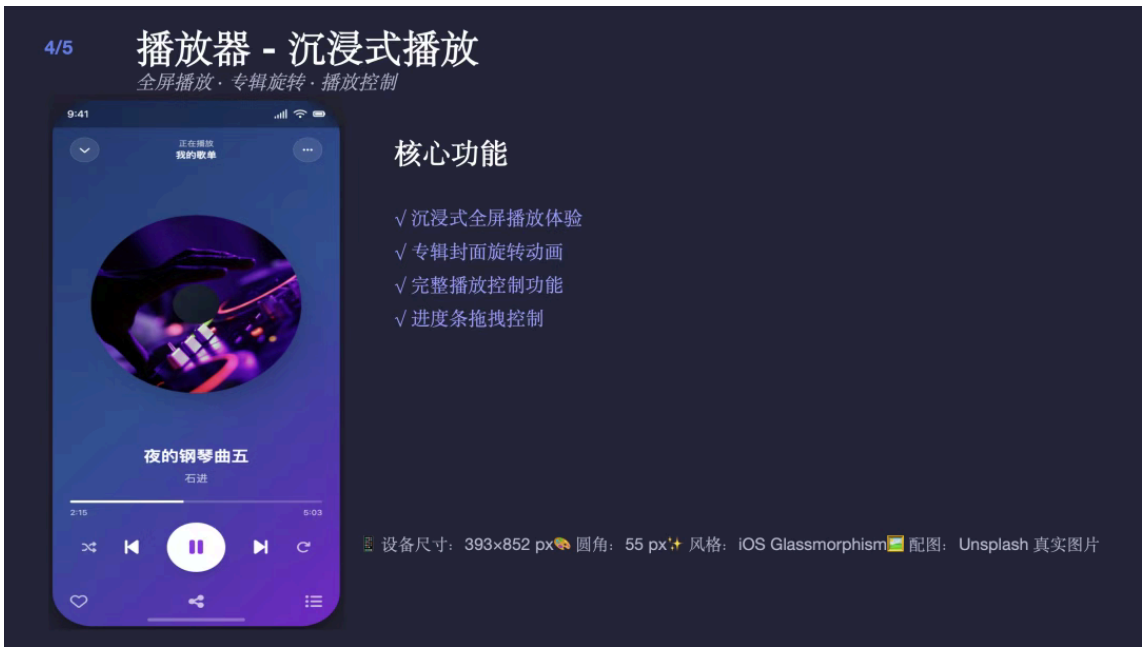
示例图：CodeBuddy PPT Skill 生成 HTML 效果图2



示例图：CodeBuddy PPT Skill 生成 PPT 发现页效果图



示例图：CodeBuddy PPT Skill 生成 PPT 播放页效果图



通过上述 Prompt 生成 HTML 和 PPT 基本已经达到可用状态，对于 MVP 产品场景而言，HTML 基本满足可用。

3.2 基于现有PPT编辑实战

3.2.1 输入 Prompt

```
#前提条件，下载好PPT 在本地，并放在 IDE 文件夹之中，可以满足添加到对话中
基于该 PPT 版本，在第 7 页后面补充一页 关于 CodeBuddy 在腾讯内部落地的情况，信息如下：
腾讯云代码助手（Tencent Cloud CodeBuddy，简称CodeBuddy），覆盖支持多款主流IDE和主流编程语言及框架，
```

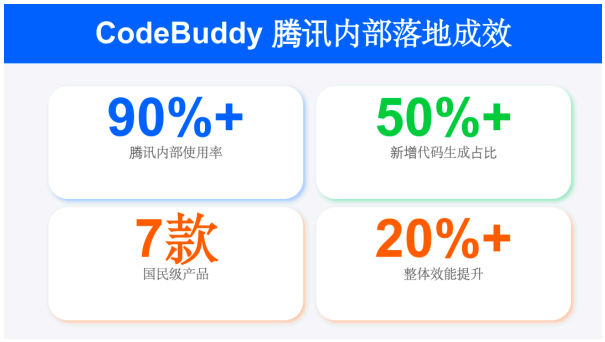
示例图：CodeBuddy 调用 Skill 进行执行交互图



示例图：CodeBuddy 调用 Skill 进行执行交互图

3.2.2 检验效果

从背景上看是完全做到了一样，在字体上 AI 没有完全对齐，比如缩短 40% 和降低 31.5% 的字体进行放大，也可能和我的 Prompt 提示词有关系，此外对于顶部标题在我的 Prompt 中没有输入，因此实现，总体来说满足需求，手工稍微改动即可使用。

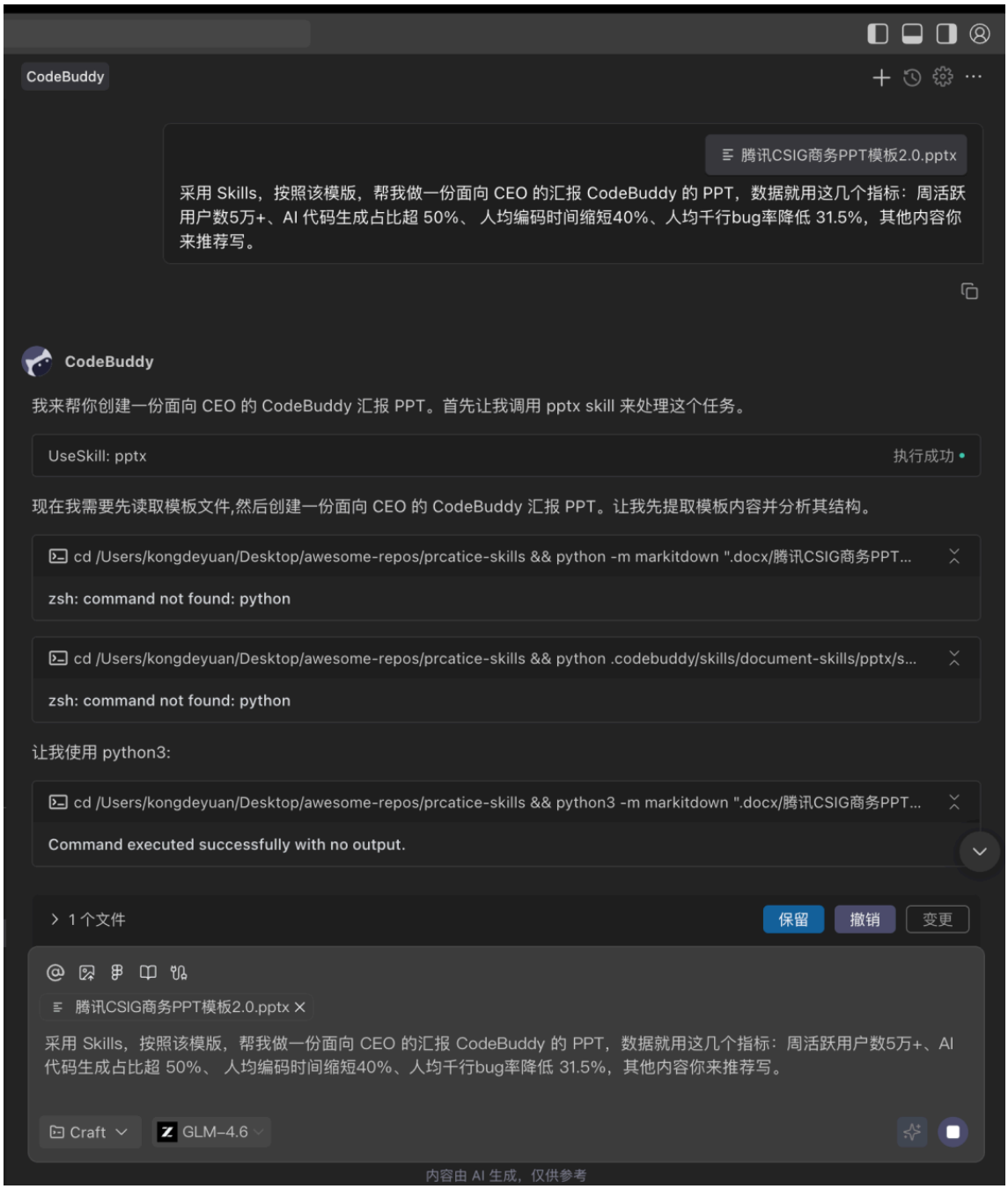


3.3 有模板场景下的PPT实战

3.3.1 输入 Prompt

```
采用 Skills，按照该模版，帮我做一份面向 CEO 的汇报 CodeBuddy 的 PPT，数据就用这几个指标：周活跃用户数
```

示例图：CodeBuddy 调用 PPT Skill + 现有模板执行交互图



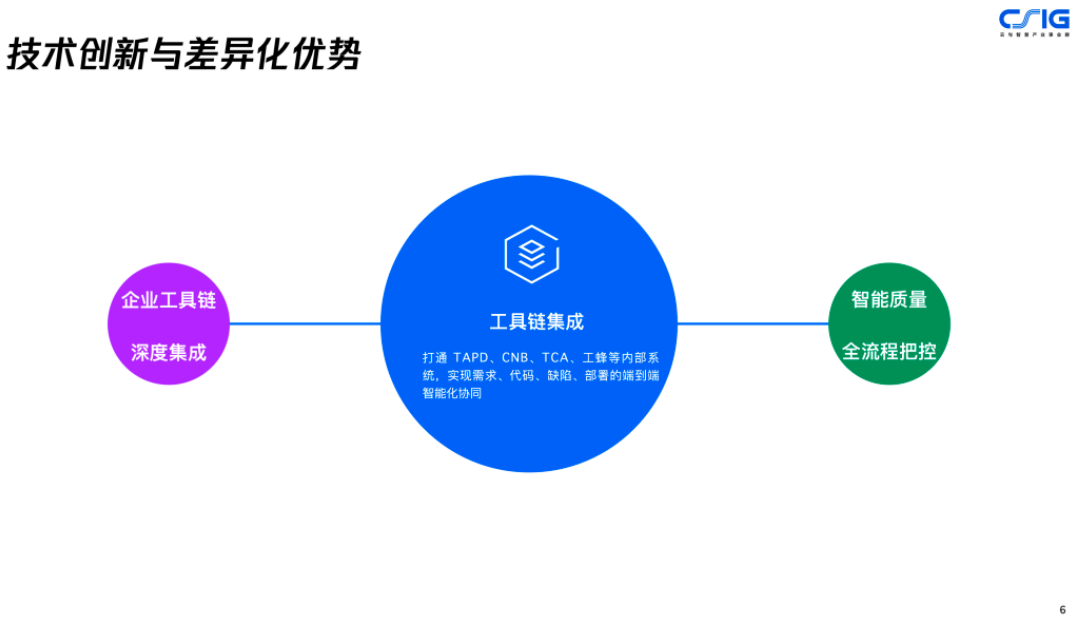
3.3.2 检验效果

这一次，坦白说，我真的看不出来是 AI 写的，令我感到惊讶！**CodeBuddy-CEO-Report.pptx**

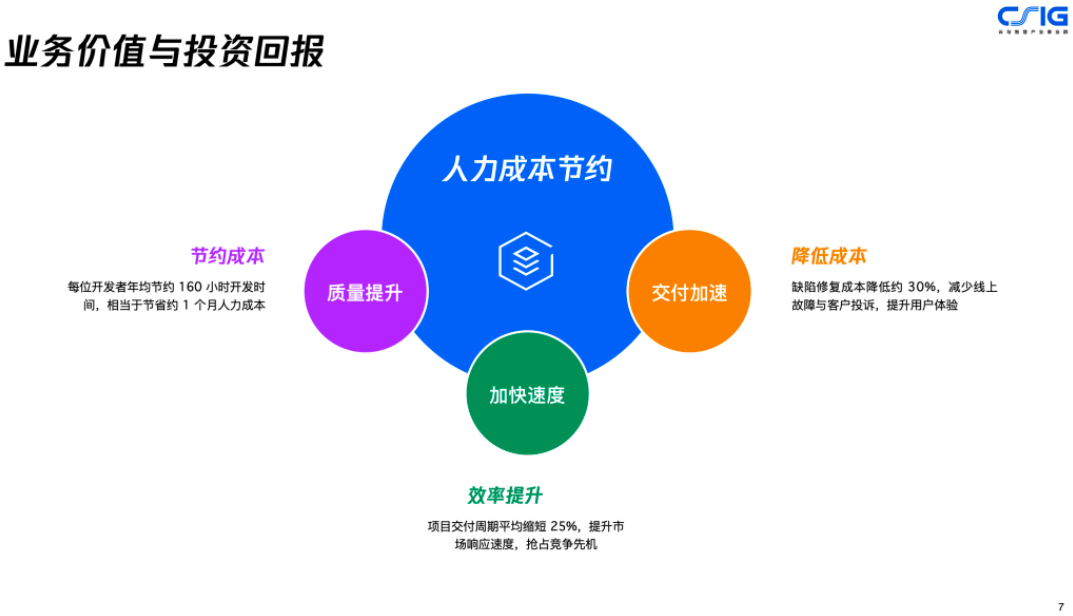
示例图：CodeBuddy 调用 PPT Skill + 腾讯云上午模板生成封面图



示例图：CodeBuddy 调用 PPT Skill + 腾讯云上午模板生成技术创新与差异化优势图



示例图：CodeBuddy 调用 PPT Skill + 腾讯云上午模板生成业务价值与投资回报图



示例图：CodeBuddy 调用 PPT Skill + 腾讯云上午模板生成未来发展规划

3.4 基于非 PPT 文档输出 PPT实战

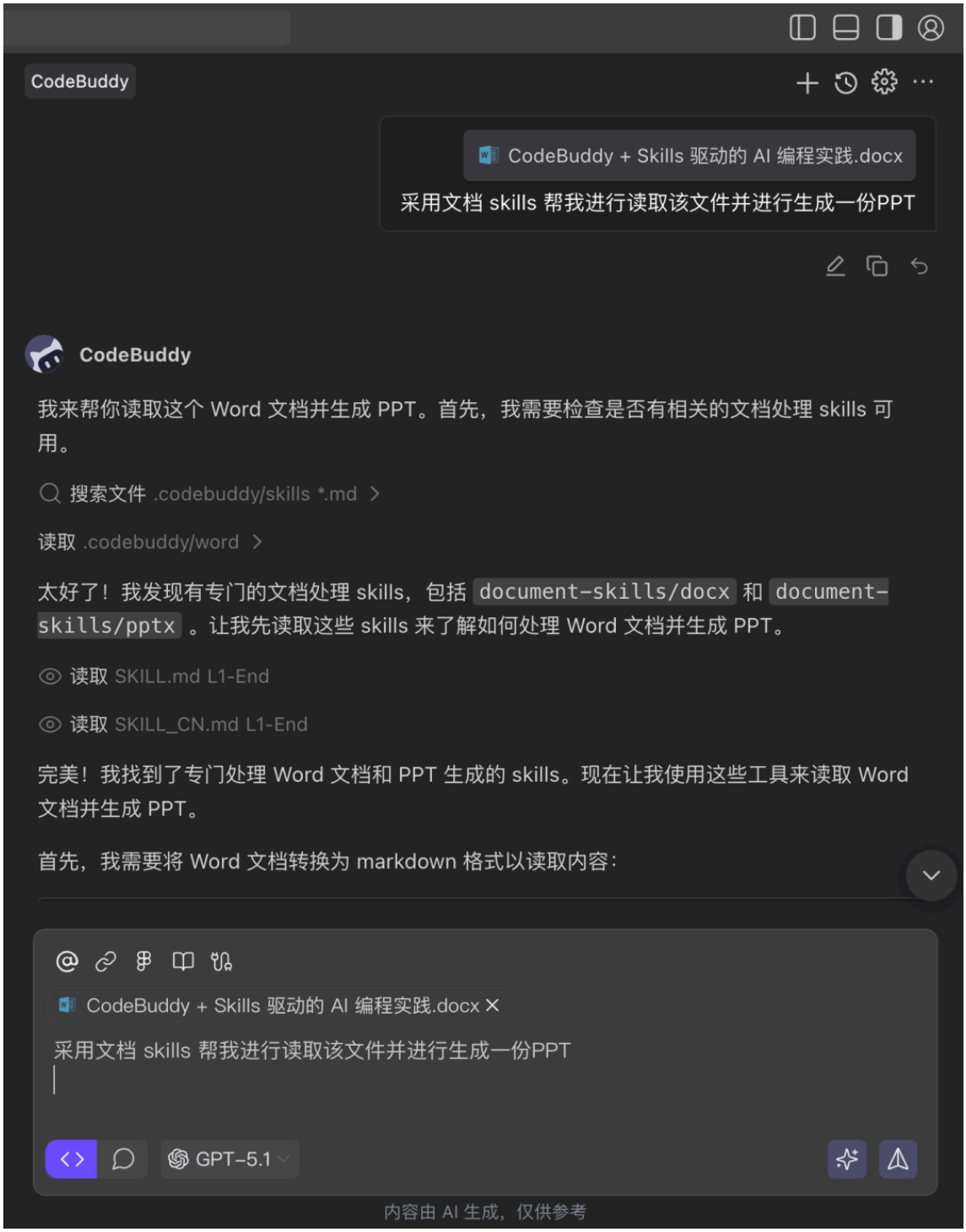
在 PPT 实战场景中，除了上述两种类型，我们还有一种类型，即比如 word 或者 md、html 等形式的文档形式，最终需要以 PPT 的方式进行对外呈现.

3.4.1 基于现有 Word 文章直接生成 PPT

以我上次在腾讯学堂分享的这份 PPT 来说，我数了一下有 27 页（占比 51%）是借助 CodeBuddy 来生成的，整个 PPT 的制作实践合计不到 4 个小时，总共 53 页**CodeBuddy 重磅升级功能详解+AI 编程实战秘籍**

对我而言，这不仅仅是一份 PPT，更是一份实战和对使用 AI 的充分信心，当时我的做法很简单，就一步：将我写的 word 文档导出该场面目录，然后输入“采用文档 skills 帮我进行读取该文件并进行生成一份PPT” Prompt

示例图：基于 docx 进行生成 PPT Prompt 及交互图



示例图：基于 docx 进行生成 PPT 总结

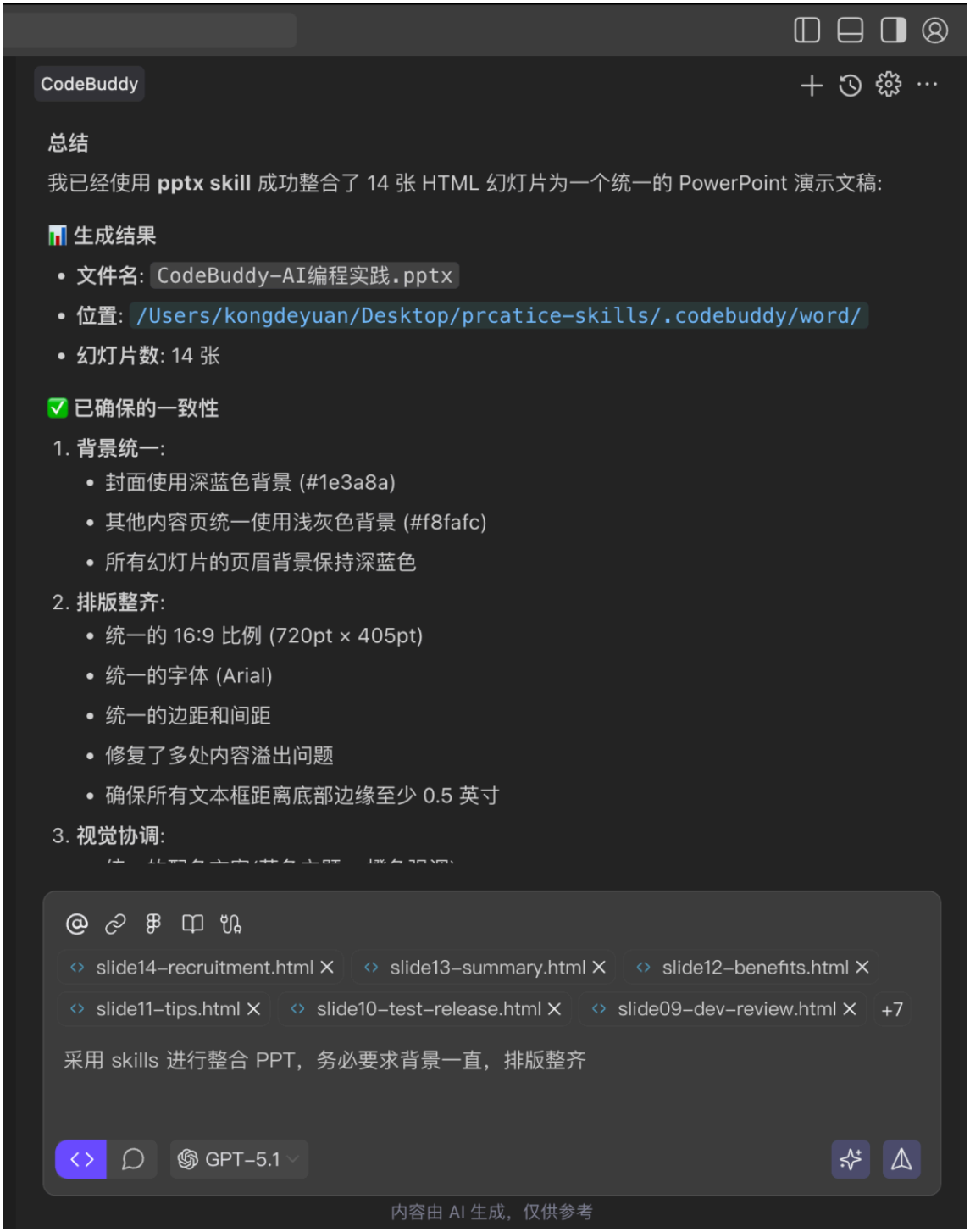


同样，基于 Hooks、Spec、CodeBuddy Code 以及最新的功能实践分享，都是基于上述方法生成，最终进行人工合成。

3.4.2 基于现有 Html 直接生成 PPT

如下为我基于《CodeBuddy AI 编程在企业团队中的落地实践与思考》文章进行 拆解生成了 14 张 HTML 介绍，最终又 12 张 HTML 进行合成一份 PPT，详见：**CodeBuddy AI 编程在企业团队中的落地实践与思考.pptx**，总体来说对我而言，超出预期。

示例图：基于 14 张 HTML 合并为一个整的 PPT


CodeBuddy
总结
我已经使用 pptx skill 成功整合了 14 张 HTML 幻灯片为一个统一的 PowerPoint 演示文稿:
📊 生成结果
文件名: CodeBuddy-AI编程实践.pptx
位置: /Users/kongdeyuan/Desktop/prcatice-skills/.codebuddy/word/
幻灯片数: 14 张
✅ 已确保的一致性
1. 背景统一:
封面使用深蓝色背景 (#1e3a8a)
其他内容页统一使用浅灰色背景 (#f8fafc)
所有幻灯片的页眉背景保持深蓝色
2. 排版整齐:
统一的 16:9 比例 (720pt × 405pt)
统一的字体 (Arial)
统一的边距和间距
修复了多处内容溢出问题
确保所有文本框距离底部边缘至少 0.5 英寸
3. 视觉协调:
slide14-recruitment.html
slide13-summary.html
slide12-benefits.html
slide11-tips.html
slide10-test-release.html
slide09-dev-review.html
+7
采用 skills 进行整合 PPT，务必要求背景一直，排版整齐
GPT-5.1
内容由 AI 生成，仅供参考

示例图： PPT 封面



示例图： PPT - 团队 AI 研发流程



示例图： PPT - AI 编程技巧指北



示例图： PPT - 总结与展望



四、实践技巧

通过上面场景和日常使用 CodeBuddy 以及写 PPT 总结一些技巧，一并分享大家。

4.1 使用 CodeBuddy的技巧

4.1.1 清晰表达需求，将任务拆解得足够正交（互不干扰、独立完整）

明确和清楚的表达你的需求这其实是一件比较难的事情，人与人之间交流大多数是有背景，但是在和 AI 打交道，AI 并不一定知道背景，只有我们清楚的表达需求并且把任务拆解的足够正交（Orthogonal），即：每个任务可以单独完成，不依赖其他任务的细节、每个任务有明确的输入、处理、输出，任务之间职责清晰，互补干扰或冲突，最后每个任务都能独立进行测试。

4.1.2 一次只改一个小目标，要聚焦

同上述一样，不要期望 AI 一次帮你实现一个大需求，尝试小步慢跑，遇到问题进行定位，持续迭代，也能让你在使用 CodeBuddy 过程中有参与感和成就感，而不是无脑的接受 AI 的输入。

4.1.3 把 AI 当导师一样看待，多轮对话进行交流

遇到不知道的地方，不认识的地方或者有疑虑，一张图丢进去问 AI，多问“为什么”、“怎么做”、“有什么坑避免”、“有什么风险”，把 AI 当做 7x 24 小时的导师一样看待，循序渐进，多轮对话交流，让 AI 协助你定位问题、分析问题、解决问题和获得新知识。

4.1.4 频繁做好备份和版本管理， 改崩了就回退

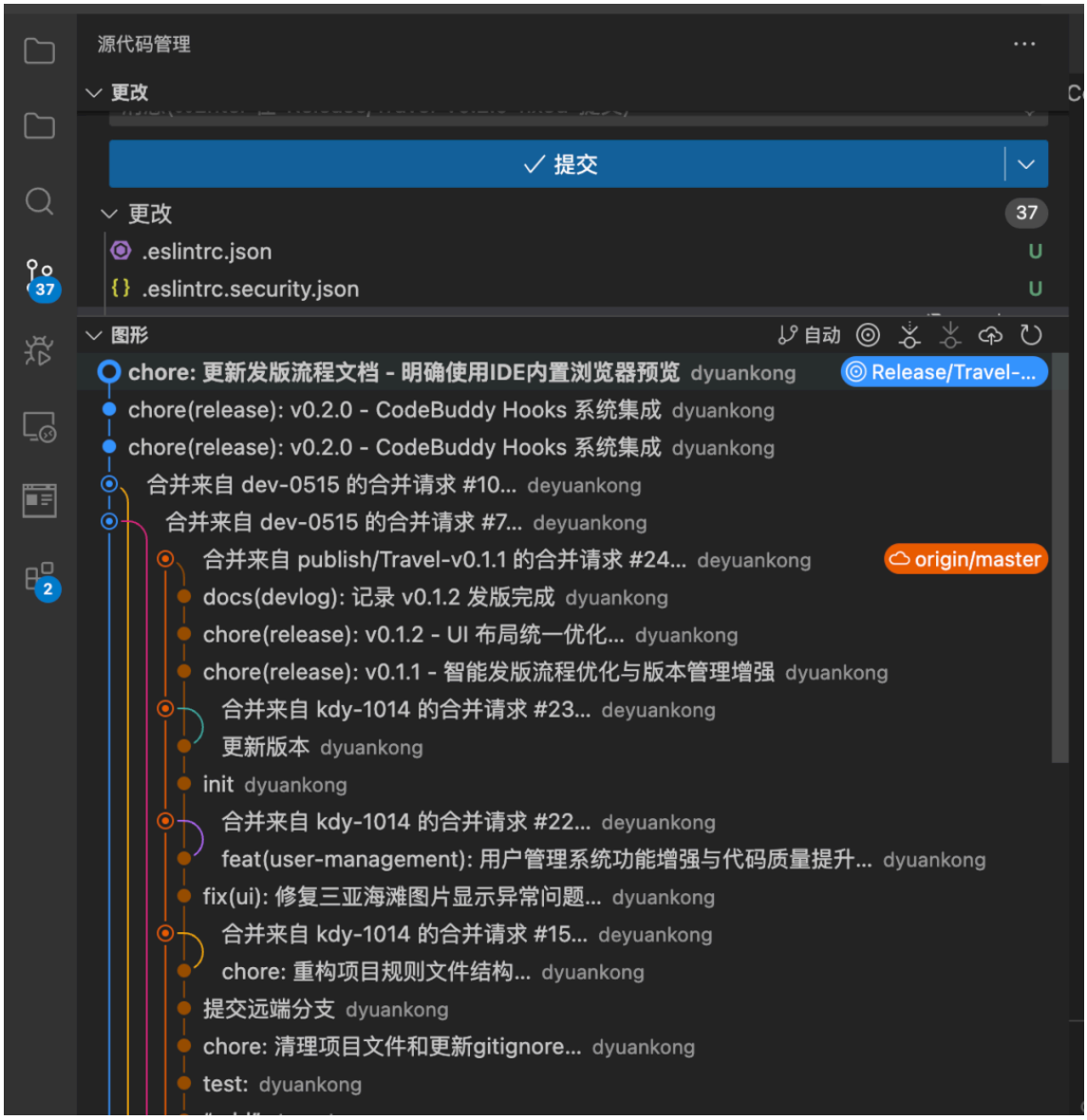
在与 AI 进行写作过程中，AI 并不一定完全按照你的意图来实现，可能正在你任务即将大功告成的时候，AI 给你来个胡乱改，导致前功尽弃，此时，你也不需要崩溃，崩了就崩了，回退回去就好，在这里特别强调的是，一定要及时做好版本化管理，频繁的提交，最好用 Git，该崩了就回退，这是在关键节点的。Git 对新人也没有那么难，就 3 条命令，每个 AI 任务后的基本操作，如果你不清楚，就把这个丢给 CodeBuddy 对话框就可以了，只要改成一一个任务就丢一遍命令。

```
git init    # 初始化新的 Git 仓库，首次使用。
git add .   #  每次有内容，就进行 add 暂存你的更改
git commit -m "添加搜索组件"  # 用描述性消息保存
```

Git 入门详见 CloudStudo 在线课程：https://cloudstudio.net/courses/28084712534933504

只要本地用了 GIt 初始化，就会有版本化管理，只要这个目录没有删除，随时可以找回你上一个版本。

示例图：基于 Git 在本地提交和版本记录



4.1.5 善用图片以及及时补充上下文细节，让 AI 不跑偏

提供具体的上下文，比如图片，一图胜千文，及时补充细节而非假设，例如需求规格约束、要求必须按照 xxx 流程、规范。

4.2 写 PPT 技巧

4.2.1 明确目标与受众，确立中心思想

开始前先问自己：谁是受众？他们需要什么信息？ 我们想传递什么信息？希望他们做什么？根据受众需求确定 PPT 的核心目标和选择场景，如汇报总结、晋升答辩、培训讲解等场景。

4.2.2 构建清晰结构框架，套用模板，活学活用

很多人在 PPT 堆里来回挪动，越改越乱，本身不是素材的问题，而是没有看懂它们之间的关系。先确定你想证明的能力/结论是什么，再选择能支撑这些能力/结论的两三条主线，让材料从堆叠变成结构。可以分三步走：

(1)第一步：搭框架

确定好上述目标和受众以及中心思想，接下来就是至少讲清楚按“为什么（Why）—怎么做（How）—有什么结果（What）”的顺序组织内容，确保有结构、有逻辑。

(2)第二步是补足关键细节

为什么这么做、难点在哪里、你解决了什么问题、怎么解决的、效果如何？这些都是让材料站得住的硬信息。细节不求多，但求能体现判断力、方法、影响范围，没有逻辑链的内容只会分散注意力。

(3)第三步是保证论证清晰

以晋升或汇报为例，本质上都是证明题。每一页内容都要回答一句话：它在证明什么、日常的细节。如果既可要也可不要，那往往说明你还没完全想明白，逻辑清晰，观众越能理解你。

(4)以场景案例为例

比如我经常写文章或者采访用户实践案例，经常会从如下角度考虑：

- **实践背景：**为什么要开发这个作品/实践案例，谈谈这个作品/实践或者分享的大背景、当下价值、意义，让用户看懂你在做什么

- **实现效果：**作品/实践案例的带来的效果，效果成绩，给业务带来的真实价值

- **面临挑战：**当时遇到什么问题、挑战、难点，说明必要性、把业务难点转为技术难点，突出门槛

- **分析问题：**利用 CodeBuddy 等主流技术以及围绕业务流程是如何解决这些问题、难点的（逐点展示拆解能力与深度思考，落到举措）

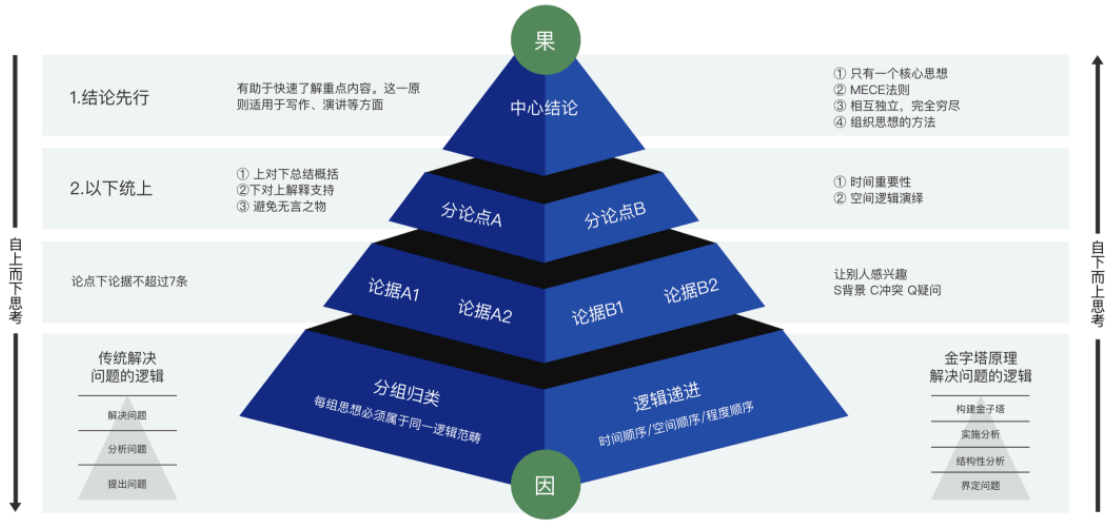
- **对比分析：**CodeBuddy 产品、方案 有何优势/亮点？，最好具体到具体的实际的 case 案例，为什么选择这个方案、这些 case ，体现对比，差异，价值突出

- **总结与展望：**提炼方法论或 VIbe Coding 技巧，让观众看到成长性以及可复用性、以及未来思考。

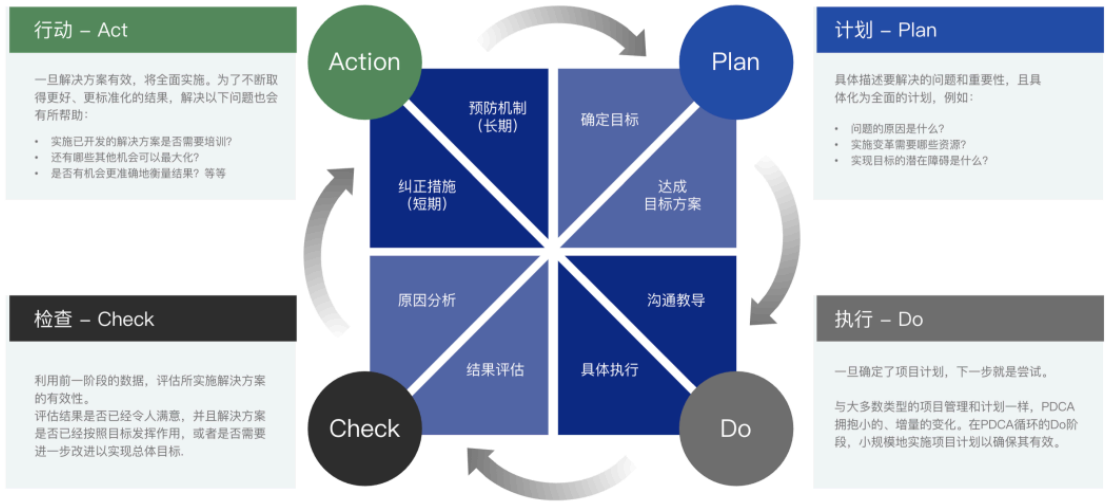
(5)参考模板

把上面几点讲清楚，比任何技巧都更靠谱，当然我们可以套用常用的逻辑表达结构 模板，比如如下模板：

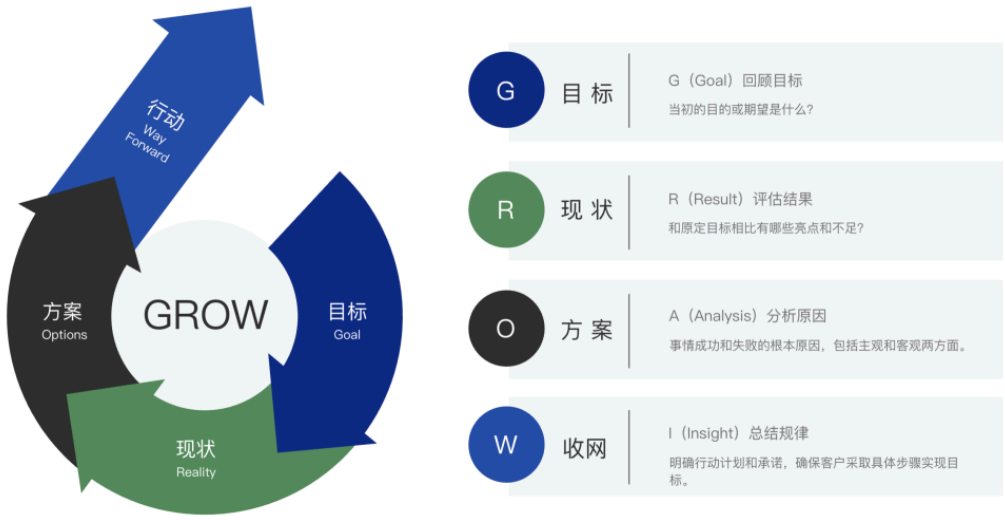
金字塔原理：



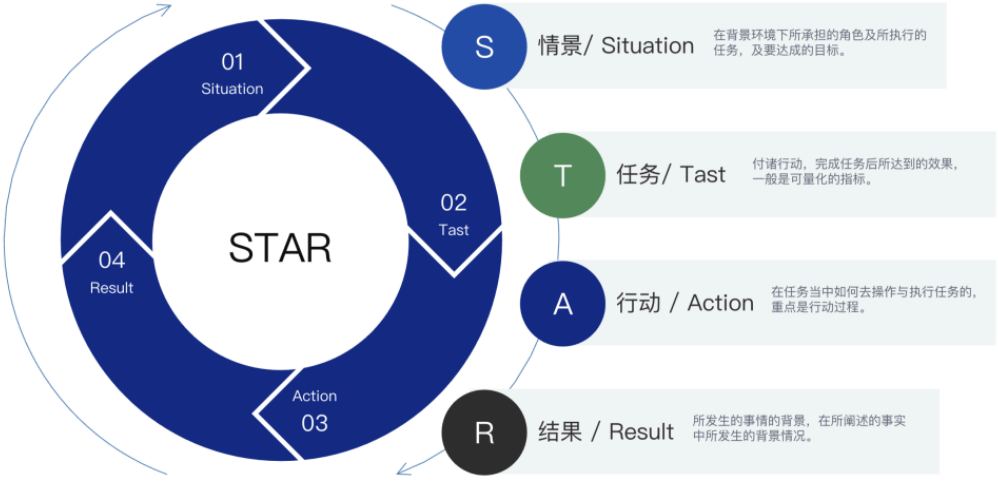
PDCA循环



GROW模型



STAR法则



除上述的模型及法则外，还有比如 SCQA模型、冰山模型、SMART 原则、沉默成本模型分析、SWOT分析法、5W2H分析法、5M1E 分析法、6S 现场管理、鱼骨分析法，这些应用场景各不一样，可以私下进行学习了解，在尾部我也把之前自己买的模板分享出来，供大家参考。

4.2.3 重点突出，突出核心卖点

针对一些重要信息，比如标题、关键论点等文字类，可以进行关键词加粗、变色或放大，要突出核心卖点，针对数据累，可以参考可视化处理，吸引用户首先关注。PPT 最好每页只突出1-2个重点，避免过度设计。

4.2.4 数据化直观表达与解读原则

对于涉及数据类的表达，至关重要，不要单纯罗列数据，单纯的数据展示没有价值，观众关心的不是数据本身，而是数据背后的解读与洞察、意义和行动建议，帮助用户更好的理解，总结实践= 关键指标 + 可视化图表 + 对比分析 + 洞察结论。以下 3 个方法助力大家用好数据。

(1)精简关键指标：避免对数据，只展示关键指标

- ❌ 避免：罗列所有数据，堆砌数字
- ✅ 推荐：只展示 3-5 个核心指标
- ⚠️ 必做：注明每个指标的定义（如"DAU： 日活跃用户数"）

(2)做好数据可视化展示，选对可视化图表

根据数据特性选择最合适的图表类型，折线图要显趋势，饼图看占比，柱状图做对比

图表类型	适用场景	示例
折线图	展示趋势变化	月度销售额走势
饼图	显示占比分布	市场份额占比
柱状图	进行数值对比	各产品线销量对比

(3)数据要有对比与解读，侧重洞察

单纯展示数据没有价值，需要加入：

- **前后对比维度**：

-时间对比（同比/环比）

-竞品对比（行业排名/对手数据）

-目标对比（实际完成率 vs 预期目标）

- **背景与洞察**：

-提供数据背景（如"疫情后线上订单占比提升"）

-给出趋势解读（如"Q2 增速放缓，需加强营销投入"）

-标注关键结论（如"核心用户留存率达 85%，行业领先"）

4.2.5 统一色彩风格与字体、行距，注意小细节

格式的一致性比花哨的效果更重要，细节统一能大幅提升专业感。具体如下：

(1)色彩与字体的克制法则

遵循"3 色 + 2 字体"的黄金组合：配色不超过 3 种（背景色、主色、辅色），搭配协调且符合主题，字体不超过 2 种（标题用无衬线体、正文保持统一），并确保字号层级分明。这种克制能让 PPT 看起来更专业，避免视觉混乱。

(2)留白与呼吸感设计

秉承"字小、留白"原则，减少每页文字量， 一般设置 1.2-1.5 倍行距避免拥挤感。适当的空白区域能让内容更易阅读，给观众视觉喘息空间，提升信息传达效率。

(3) 对齐一致性原则

所有文字、图片、图表统一采用左对齐或居中对齐，杜绝参差不齐的布局。细节的对齐统一虽然不起眼，却能潜移默化地提升整体专业感，体现制作者的用心程度。



腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

689篇原创内容

公众号

五、深度剖析

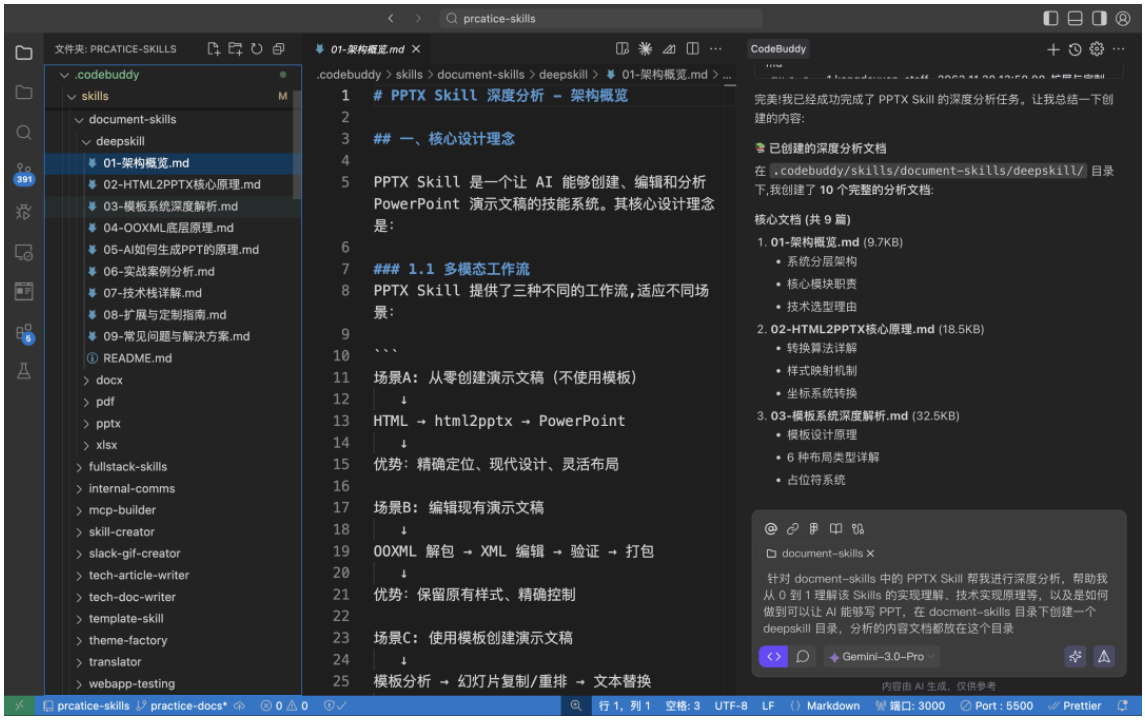
基于上述场景实战，我重点分析以下下该 ppt skill 实现的内部逻辑原理， 因此我借助

CodeBuddy 帮我进行理解，最终生成了 10 份 MD, ， 接下来我进行提炼和汇总核心内容。

示例图： 使用 CodeBuddy 针对 PPTX Skill 进行深度理解和分析交互图



示例图： 生成 MD 文档示例图



5.1 重新认识pptskill

document-skills pptx 是一个专门用于处理 PowerPoint 演示文稿的高级技能工具，它 能让 AI 具备从创建、编辑到分析的完整的 PPT 处理能力。该技能采用多层技术架构，结合了 HTML 到 PPT 的转换、OOXML 操作、模板系统等多种技术手段，最终实现 PPT 输出。

5.2 多模态工作流与核心架构

5.2.1 三种多模态工作流

PPTX Skill 提供了三种不同的工作流,适应不同场景：包括不使用模版、编辑现有 PPT ， 以及使用模板创建 PPT。

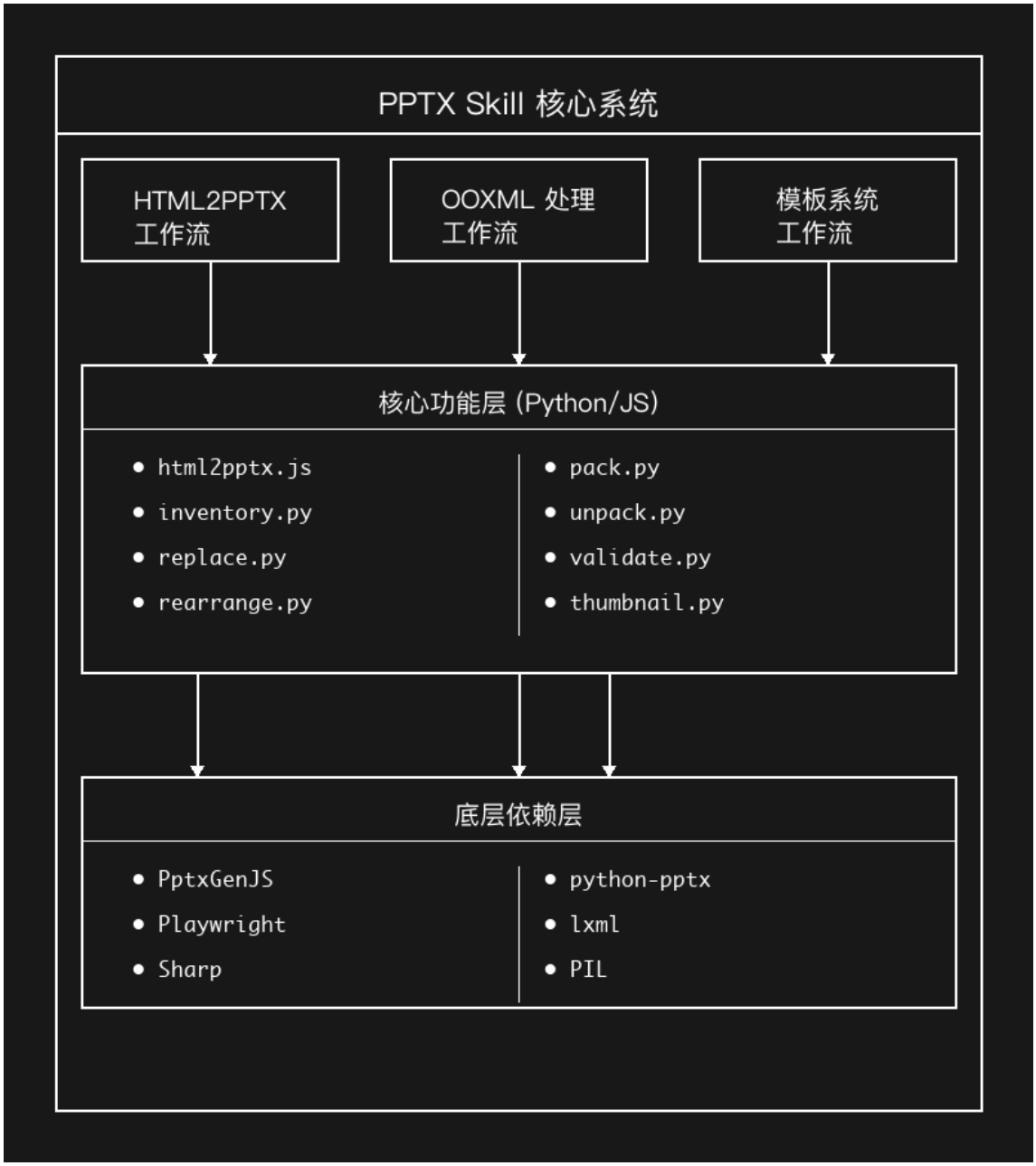
```
场景A: 从零创建演示文稿 (不使用模板)
  ↓
HTML → html2pptx → PowerPoint
  ↓
优势: 精确定位、现代设计、灵活布局

场景B: 编辑现有演示文稿
  ↓
OOXML 解包 → XML 编辑 → 验证 → 打包
  ↓
优势: 保留原有样式、精确控制

场景C: 使用模板创建演示文稿
  ↓
模板分析 → 幻灯片复制/重排 → 文本替换
  ↓
优势: 保持品牌一致性、快速生成
```

PPT Skill 核心架构图：

示例图：PPT Skill三种多模态工作流核心系统工作图



5.2.2 工作流 1： html2pptx 工作流

应用场景：从0到1创建PPT

作用: 将 HTML 幻灯片精确转换为 PowerPoint

技术实现：

```
HTML：通过前端 HTML 实现效果
Playwright：浏览器自动化引擎，用于渲染 HTML 内容
Sharp：图像处理库，用于SVG栅格化和图标处理
PptxGenJS：JavaScript PPT生成库
React Icons：图标组件库
```

核心工作流程

```
1.  AI创建 HTML 幻灯片（带尺寸约束），设置精确尺寸，使用 <p>, <h1>-<h6>, <ul>, <ol> 包裹所有文本
   ↓
2. Playwright 启动无头浏览器（Chromium） 渲染 HTML，
   ↓
3. Dom 提取元素位置和样式、定位
   ↓
4. CSS 样式解析与转换
   ↓
5. 位置精确计算（px → pt → inches 转换）
   ↓
6. 转换为 PptxGenJS 对象
   ↓
7. 样式应用与尺寸、内容溢出等验证
   ↓
8. 生成 .pptx 文件
```

核心算法：

```
// HTML 尺寸约束
```javascript
// html2pptx.js - getBodyDimensions()
const bodyDimensions = await page.evaluate(() => {
  const body = document.body;
  const style = window.getComputedStyle(body);

return {
    width: parseFloat(style.width),    // 720px (16:9)
    height: parseFloat(style.height),  // 405px
    scrollWidth: body.scrollWidth,     // 检测溢出
    scrollHeight: body.scrollHeight
  };
});
```

// 单位转换
```javascript
const PT_PER_PX = 0.75;   // 1px = 0.75pt
```
```

```
const PX_PER_IN = 96;      // 1inch = 96px

// RGB 转十六进制 (PowerPoint 格式)
const rgbToHex = (rgbStr) => {
  // "rgb(255, 0, 0)" → "FF0000"
  const match = rgbStr.match(/rgb\((\d+),\s*(\d+),\s*(\d+)\)/);
return match.slice(1)
    .map(n => parseInt(n).toString(16).padStart(2, '0'))
    .join('');
};
```

关键特性：

■ **精确位置控制：**实现点级精度转化，转化公式: 1px = 0.75pt，1英寸 = 96px = 72pt = 914400 EMU

■ **溢出检测算法和自动验证：**检查 HTML 尺寸与 PPT 布局的匹配度、文本位置是否溢出或超出边界、CSS格式兼容性，要求底部保留0.5英寸边距

■ **占位符机制:** HTML 中用

标记位置、html2pptx 提取占位符的坐标、用 PptxGenJS API 在精确位置插入图表

5.2.3 工作流 2： OOXML 处理工作流

应用场景： 保持原有设计不变，精准编辑现有 PPT 文件，批量文本替换

作用: 直接操作 PowerPoint 的底层 XML 格式

技术实现：

-**ZIP文件处理：**PPTX 本质是 ZIP 压缩包，包含 XML 文件、关系文件（_rels）、媒体资源

-**XML结构操作：**直接修改 slideN.xml、presentation.xml等核心文件

-**遵循标准：**确保 XML结构符合Office Open XML ISO/IEC 29500、ECMA-376、开放规范

核心工作流程

```
1. 解包 .pptx → XML 文件
   ↓
2. 编辑 XML（文本、样式、结构）
   ↓
3. XSD Schema 验证
   ↓
4. 打包 → 新的 .pptx 文件
```

PPTX 文件结构解析

```
presentation.pptx (ZIP 压缩包)
├── [Content_Types].xml          # 文件类型定义
├── _rels/        # 根关系
|   └── .rels    # 包关系
├── docProps/     # 文档属性
|   ├── app.xml # 应用属性（幻灯片数）
|   └── core.xml# 核心属性（作者、标题）
└── ppt/          # 演示文稿内容
    ├── presentation.xml         # 演示文稿主文件
    ├── _rels/   # 演示文稿关系
    |   └── presentation.xml.rels
    ├── slides/   # 幻灯片
    |   ├── slide1.xml
    |   ├── slide2.xml
    |   └── _rels/   # 幻灯片关系
    |       ├── slide1.xml.rels
    |       └── slide2.xml.rels
    ├── slideLayouts/# 幻灯片布局
    |   ├── slideLayout1.xml
    |   └── _rels/
    ├── slideMasters/# 母版幻灯片
    |   ├── slideMaster1.xml
    |   └── _rels/
    ├── theme/    # 主题
    |   └── theme1.xml
    ├── media/    # 媒体文件
    |   ├── image1.png
    |   └── image2.jpg
    └── notesSlides/ # 演讲者备注（可选）
        └── notesSlide1.xml
```

核心算法：

- **解包工具算法**

```
#unpack.py - 解包工具

```python
#!/usr/bin/env python3
"""
解包 Office 文件到目录

原理：
```
```

```
1. PPTX 本质是 ZIP 文件
2. 使用 zipfile 模块解压
3. 保持目录结构
"""

import zipfile
from pathlib import Path

def unpack_document(office_file, output_dir):
    """
    解包 Office 文件

    Args:
        office_file: .pptx/.docx/.xlsx 文件
        output_dir: 输出目录
    """
    output_dir = Path(output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)

    with zipfile.ZipFile(office_file, 'r') as zf:
        zf.extractall(output_dir)

    print(f"Unpacked {office_file} to {output_dir}")

# 使用
if __name__ == '__main__':
    unpack_document('presentation.pptx', 'unpacked/')
```

注意事项:
- 解包后的 XML 可能是压缩的（无换行）
- 需要格式化 XML 以便编辑
- 保留文件权限和时间戳

- **XSD Schema验证算法**

XSD Schema 验证:

```python
def validate_against_xsd(self):
    """
    使用 XSD Schema 验证 XML 结构

    Schema 位置:
    • ooxml/schemas/ISO-IEC29500-4_2016/
    • pml.xsd（演示文稿）
```

```
    • dml-main.xsd (绘图)
    """
    # 加载 XSD Schema
    pml_schema = lxml.etree.XMLSchema(
        lxml.etree.parse('ooxml/schemas/.../pml.xsd')
    )

    errors = []
    for xml_file in self.xml_files:
        doc = lxml.etree.parse(str(xml_file))

        if not pml_schema.validate(doc):
for error in pml_schema.error_log:
errors.append(
    f"{xml_file}:{error.line} {error.message}"
)

    return len(errors) == 0
```

5.2.4 工作流 3： 模板系统工作流

应用场景：基于现有模板快速生成 PPT

作用: 直接基于现有 PPT 模板分析

技术实现：基于模板+ 算法实现模板分析、创建内容大纲与清单、使用算法复制和排序 PPT

核心工作流程

```
# 工作流程
1. 分析模板（缩略图 + 文本提取）
   ↓
2. 创建清单 & 内容映射（内容 → 模板幻灯片）
   ↓
3. 复制/重排幻灯片
   ↓
4. 文本内容提取与替换
   ↓
5. 视觉验证输出
```

核心算法：

- **内容提取算法**

```
### inventory.py - 智能文本分析
```

核心数据结构

```python
@dataclass
class ParagraphData:
    """段落数据"""
    text: str
    bullet: bool = False
    level: Optional[int] = None
    alignment: Optional[str] = None
    space_before: Optional[float] = None
    space_after: Optional[float] = None
    font_name: Optional[str] = None
    font_size: Optional[float] = None
    bold: Optional[bool] = None
    italic: Optional[bool] = None
    color: Optional[str] = None
    line_spacing: Optional[float] = None

@dataclass
class ShapeData:
    """形状数据"""
    left: float# 位置（英寸）
    top: float
    width: float
    height: float
    placeholder_type: Optional[str]  # TITLE, BODY, etc.
    paragraphs: List[ParagraphData]
    overflow: Optional[dict]  # 溢出信息
```

文本提取算法

```python
def extract_text_inventory(pptx_path):
    """
    提取所有文本形状及其属性

    流程：
    1. 遍历所有幻灯片
    2. 递归处理 GroupShape
    3. 提取形状位置和文本
    4. 按视觉位置排序
    5. 检测溢出和重叠
    """
    prs = Presentation(pptx_path)
    inventory = {}
```

```
    for slide_idx, slide in enumerate(prs.slides):
        # 收集所有有效形状（含组内形状）
        shapes = []
        for shape in slide.shapes:
shapes.extend(
collect_shapes_with_absolute_positions(shape)
)

        # 转换为 ShapeData
        shape_data_list = [
ShapeData(
swp.shape,
swp.absolute_left,
swp.absolute_top,
slide
)
for swp in shapes
        ]

        # 按视觉位置排序
        sorted_shapes = sort_shapes_by_position(shape_data_list)

        # 分配稳定 ID
        for idx, shape_data in enumerate(sorted_shapes):
shape_data.shape_id = f"shape-{idx}"

        # 检测重叠
        detect_overlaps(sorted_shapes)

        inventory[f"slide-{slide_idx}"] = {
sd.shape_id: sd for sd in sorted_shapes
        }

    return inventory
```

- **智能文本替换算法**

```
### 3.4 replace.py - 智能文本替换

#### 替换流程

```python
def apply_replacements(pptx_file, json_file, output_file):
    """
    应用文本替换
```
```

```
    流程：
    1. 加载演示文稿
    2. 提取清单（获取形状引用）
    3. 验证替换 JSON
    4. 清除所有形状的文本
    5. 应用新文本（仅替换 JSON 中指定的）
    6. 检测溢出变化
    7. 保存并验证

    关键设计决策：
    • 默认清除所有文本（避免残留）
    • 仅替换 JSON 中有 "paragraphs" 的形状
    • 验证溢出是否恶化
    """
    prs = Presentation(pptx_file)

    # 提取清单（获取形状对象引用）
    inventory = extract_text_inventory(Path(pptx_file), prs)

    # 加载替换数据
    with open(json_file) as f:
        replacements = json.load(f)

    # 验证
    errors = validate_replacements(inventory, replacements)
    if errors:
        raise ValueError(f"Invalid replacement JSON")

    # 处理每个形状
    for slide_key, shapes_dict in inventory.items():
        for shape_key, shape_data in shapes_dict.items():
shape = shape_data.shape
text_frame = shape.text_frame

# 清除文本
text_frame.clear()

# 检查是否有替换
replacement = replacements.get(slide_key, {}).get(shape_key, {})
if'paragraphs' not in replacement:
continue# 保持空白

# 应用新段落
for i, para_data in enumerate(replacement['paragraphs']):
if i == 0:
    p = text_frame.paragraphs[0]
else:
    p = text_frame.add_paragraph()
```

```
apply_paragraph_properties(p, para_data)

    # 验证输出
    prs.save(output_file)
    validate_output(output_file)
```

- **递归处理算法**

```
#### GroupShape 递归处理

```python
def collect_shapes_with_absolute_positions(
    shape,
    parent_left=0,
    parent_top=0
):
    """
    递归处理组形状，计算绝对位置

    挑战：
    • 组内形状的坐标相对于组
    • 需要累加所有父级偏移
    • 支持嵌套组

    解决：
    ├── 检测是否为 GroupShape
    ├── 累加父级偏移
    ├── 递归处理子形状
    └── 返回绝对位置
    """
    if hasattr(shape, 'shapes'):  # GroupShape
        group_left = shape.left
        group_top = shape.top

        # 累加偏移
        abs_left = parent_left + group_left
        abs_top = parent_top + group_top

        # 递归处理子形状
        result = []
        for child in shape.shapes:
result.extend(
collect_shapes_with_absolute_positions(
    child, abs_left, abs_top
```
```

```
)
)
        return result

    # 普通形状
    if is_valid_shape(shape):
        return [ShapeWithPosition(
shape=shape,
absolute_left=parent_left + shape.left,
absolute_top=parent_top + shape.top
        )]

    return []
```

视觉排序算法

视觉排序算法

```python
def sort_shapes_by_position(shapes):
    """
    按视觉位置排序（从上到下，从左到右）

    算法：
    1. 首先按 top 位置排序
    2. 分组为"行"（垂直容差 0.5"）
    3. 每行内按 left 位置排序

    为什么需要容差？
    • 设计师可能手动对齐，不完全精确
    • 0.5" ≈ 36pt，合理的行高差异
    """
    if not shapes:
        return shapes

    # 按 top 排序
    shapes = sorted(shapes, key=lambda s: (s.top, s.left))

    # 分行
    result = []
    row = [shapes[0]]
    row_top = shapes[0].top

    for shape in shapes[1:]:
        if abs(shape.top - row_top) <= 0.5:
row.append(shape)
```

```
        else:
# 当前行按 left 排序
result.extend(sorted(row, key=lambda s: s.left))
row = [shape]
row_top = shape.top

    # 最后一行
    result.extend(sorted(row, key=lambda s: s.left))

    return result
```

关键特性：

```
幻灯片：命名 规范，以 "slide-0"、"slide-1" 等定义。
形状：按视觉位置排序（从上到下，从左到右）为 "shape-0"、"shape-1" 等。
占位符类型：TITLE、CENTER_TITLE、SUBTITLE、BODY、OBJECT 或 null
默认字体大小：从布局占位符提取的 default_font_size（点）（如果可用）
幻灯片编号已过滤：具有 SLIDE_NUMBER 占位符类型的形状自动从清单中排除
间距：space_before、space_after 和 line_spacing（点）（仅在设置时包含）
颜色：技能内置18种专业配色方案：RGB 的 color（例如 "FF0000"），主题颜色的 theme_color（例如 "DARK_
属性：输出中仅包含非默认值
```

5.3 PPTSkill系统设计

5.3.1 设计原则

关键：在创建任何演示文稿之前，分析内容并选择适当的设计元素：

1.**考虑主题内容**：这个演示文稿是关于什么的？它暗示什么基调、行业或情绪？

2.**检查品牌**：如果用户提到公司/组织，考虑他们的品牌颜色和标识

3.**匹配内容的调色板**：选择反映主题的颜色

4.**说明你的方法**：在编写代码之前解释你的设计选择

5.3.2 设计哲学

(1) "HTML 优先"设计

核心思想: 让 AI 使用熟悉的 HTML/CSS 创建演示文稿

```
优势：
✓ AI 已精通 HTML/CSS
✓ Flexbox 布局强大
✓ 易于验证和调试
```

```
✓ 所见即所得

代价：
✗ 需要转换层
✗ 某些 PPT 特性受限
```

(2)"验证优先"设计

核心思想: 尽早发现问题，避免生成损坏文件

```
# 多层验证
1. HTML 阶段：尺寸、溢出
2. 转换阶段：元素支持度
3. XML 阶段：Schema、关系
4. 输出阶段：文件完整性
```

(3)"工具链化"设计

核心思想: 每个脚本专注单一职责

```
thumbnail.py     → 可视化
inventory.py     → 分析
rearrange.py     → 重组
replace.py       → 替换
validate.py      → 验证
pack.py          → 打包
```

5.3.3 技能内置18种专业配色和多种字体方案选择

(1)内置18 种专业配色方案

```
经典蓝：深海军蓝（#1C2833）、石板灰（#2E4053）、银色（#AAB7B8）、米白色（#F4F6F6）
青绿与珊瑚：青绿（#5EA8A7）、深青绿（#277884）、珊瑚（#FE4447）、白色（#FFFFFF）
大胆红：红色（#C0392B）、亮红（#E74C3C）、橙色（#F39C12）、黄色（#F1C40F）、绿色（#2ECC71）
温暖腮红：紫灰（#A49393）、腮红（#EED6D3）、玫瑰（#E8B4B8）、奶油（#FAF7F2）
勃艮第奢华：勃艮第（#5D1D2E）、深红（#951233）、铁锈（#C15937）、金色（#997929）
深紫与翡翠：紫色（#B165FB）、深蓝（#181B24）、翡翠（#40695B）、白色（#FFFFFF）
奶油与森林绿：奶油（#FFE1C7）、森林绿（#40695B）、白色（#FCFCFC）
粉红与紫色：粉红（#F8275B）、珊瑚（#FF574A）、玫瑰（#FF737D）、紫色（#3D2F68）
青柠与梅子：青柠（#C5DE82）、梅子（#7C3A5F）、珊瑚（#FD8C6E）、蓝灰（#98ACB5）
黑金：金色（#BF9A4A）、黑色（#000000）、奶油（#F4F6F6）
鼠尾草与陶土：鼠尾草（#87A96B）、陶土（#E07A5F）、奶油（#F4F1DE）、炭灰（#2C2C2C）
炭灰与红：炭灰（#292929）、红色（#E33737）、浅灰（#CCCBCB）
活力橙：橙色（#F96D00）、浅灰（#F2F2F2）、炭灰（#222831）
```

森林绿：黑色（#191A19）、绿色（#4E9F3D）、深绿（#1E5128）、白色（#FFFFFF）

复古彩虹：紫色（#722880）、粉红（#D72D51）、橙色（#EB5C18）、琥珀（#F08800）、金色（#DEB600）

复古大地：芥末黄（#E3B448）、鼠尾草（#CBD18F）、森林绿（#3A6B35）、奶油（#F4F1DE）

海岸玫瑰：旧玫瑰（#AD7670）、海狸（#B49886）、蛋壳（#F3ECDC）、灰绿（#BFD5BE）

橙与青绿：浅橙（#FC993E）、灰青绿（#667C6F）、白色（#FCFCFC）

(2)多种字体使用场景的字体

- Arial（无衬线，通用，几乎所有操作系统预装，适用：正文、标题，安全性最高的选择。）
- Helvetica（无衬线，Mac首选/适合印刷，适用：专业演示文稿，现代风格设计。）
- Times New Roman（衬线，正式，学术性强，适用：正式报告，学术论文，传统文档。）
- Georgia（衬线，专为屏幕阅读，适用：长篇屏幕阅读，电子书。）
- Courier New（等宽，打印机风格代码，适用：代码片段，技术文档，强调数据对齐。）
- Verdana（无衬线，屏幕优化，宽间距，适用：低分辨率屏幕，小字号文本。）

5.3.4 布局优化算法保障最佳布局

```
/**
 * 优化布局函数 - 根据内容智能选择最佳布局
 * @param {Object} content - 需要布局的内容
 * @returns {Object} - 布局配置对象
 */
function optimizeLayout(content) {
if (hasChartOrTable(content)) {
    // 使用两列布局：文本 + 图表
    return {
      type: 'two-column',
      ratio: isTextHeavy(content) ? '40/60' : '60/40',
      text_column: extractTextContent(content),
      visual_column: extractVisualContent(content)
    };
  } else {
    // 使用单列布局
    return {
      type: 'single-column',
      content: structuredContent(content)
    };
  }
}
```

5.3.5 基本要求约束

- ✅ 在编写代码之前说明你基于内容的设计方法
- ✅ 仅使用网页安全字体：Arial、Helvetica、Times New Roman、Georgia、Courier New、Verdana、Tahom
- ✅ 通过大小、粗细和颜色创建清晰的视觉层次

✅ 确保可读性：强对比度、适当大小的文本、清晰的对齐

✅ 保持一致性：在幻灯片之间重复模式、间距和视觉语言

5.4 性能优化与错误处理

并发处理机制以及错误恢复策略

■ 渐进式降级：HTML转换失败时创建基础幻灯片

■ 批量验证：一次报告所有验证错误，避免多次反复

■ 备份机制：编辑前自动创建备份文件

■ 回滚能力：操作失败时恢复原始状态

并发处理机制算法

```
sync function processSlidesParallel(htmlFiles, pptx) {
    const concurrency = 3; // 并发数量限制
    console.log(`🚀 Starting parallel processing of ${htmlFiles.length} slides (Concurren

    const chunks = chunkArray(htmlFiles, concurrency);

    for (let i = 0; i < chunks.length; i++) {
        const chunk = chunks[i];
        console.log(`\n📦 Processing batch ${i + 1}/${chunks.length} (${chunk.length} fil

        const promises = chunk.map(htmlFile =>
html2pptx(htmlFile, pptx)
// 成功回调：注入文件名以便日志记录
.then(res => {
    // 如果返回值是对象，合并 file 属性；否则返回包含 result 和 file 的新对象
    return (typeof res === 'object' && res !== null)
        ? { ...res, file: htmlFile }
        : { result: res, file: htmlFile };
})
// 失败回调：捕获错误，防止 Promise.all 整体失败
.catch(error => ({
    error: error.message,
    file: htmlFile
}))
        );

        const results = await Promise.all(promises);
        handleProcessingResults(results);
    }
```

```
    console.log('\n✨ All processing completed.');
}
```

5.5 质量保证系统

多层验证机制

■ HTML验证：检查标签嵌套、CSS合规性、尺寸限制

■ 布局验证：检查元素位置、边距要求、对齐规范

■ 内容验证：检查文本溢出、字体兼容性、颜色对比度

■ 输出验证：检查PPT文件完整性、可读性

视觉质量检查算法

```python
def generate_thumbnail_grid(pptx_path, output_prefix, cols=5):
    """生成缩略图网格用于质量检查"""
    try:
        # 1. 转换PPT为PDF
        pdf_path = convert_pptx_to_pdf(pptx_path)

        # 2. 将PDF转换为图片
        images = convert_pdf_to_images(pdf_path)

        # 3. 创建网格布局
        grid = create_image_grid(images, cols)

        # 4. 保存检查结果
        output_filename = f"{output_prefix}.jpg"
        grid.save(output_filename, quality=85)
        print(f"✅ 检查结果已保存: {output_filename}")

    except Exception as e:
        print(f"❌ 流程中断: {e}")
        sys.exit(1)
```

5.6优势与局限性

5.6.1技术优势

1.**多模态灵活支持** 提供三种开发模式，满足不同场景需求：

a.**HTML 模板渲染**：所见即所得，开发效率高。

b.**OOXML 底层操作**：精细控制文档结构，适合深度定制。

c.**存量模板复用**：基于现有 PPT 资产快速生成，降低设计成本。

2.**像素级高精转换** 内置高精度坐标计算引擎，确保从 Web 布局到 PPT 元素的像素级对齐，最大程度保持视觉一致性。

3.**企业级批量处理** 架构专为高并发场景优化，能够稳定支撑大规模幻灯片的自动化生成与批量修改任务。

4.**稳健的架构设计**

a.**模块化扩展**：松耦合设计，便于快速集成新功能。

b.**质量门禁**：引入多层验证机制，确保最终输出文件的格式合规与内容准确。

5.6.2技术局限

1.**渲染能力边界**

a.**样式限制**：暂不支持 CSS 渐变、复杂动画特效及部分极度复杂的 HTML 嵌套布局。

b.**还原损耗**：极个别特殊的 Web 布局在转换为静态 PPT 时可能存在细微差异。

2.**资源与性能约束**

a.**字体依赖**：主要支持 Web 安全字体，对非标字体的兼容性受限。

b.**性能开销**：在处理超大规模（如千页级）文档时，生成耗时会显著增加，需预留计算资源。

六、 总结

最后，希望大家确实摆脱 PPT 之痛，每一场汇报、晋升、培训都能很顺利，也期待大家能够使用 CodeBuddy Skills 做出自己精美的原型、作品和 PPT，以及各种疑难问题，欢迎大家加入 CodeBuddy 交流群交流。

腾讯Q币福利限量抽奖领取！

抽奖助手

抽奖



小程序

腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
689篇原创内容

公众号

阅读 9159

留言

写留言

还没有留言，来写首评